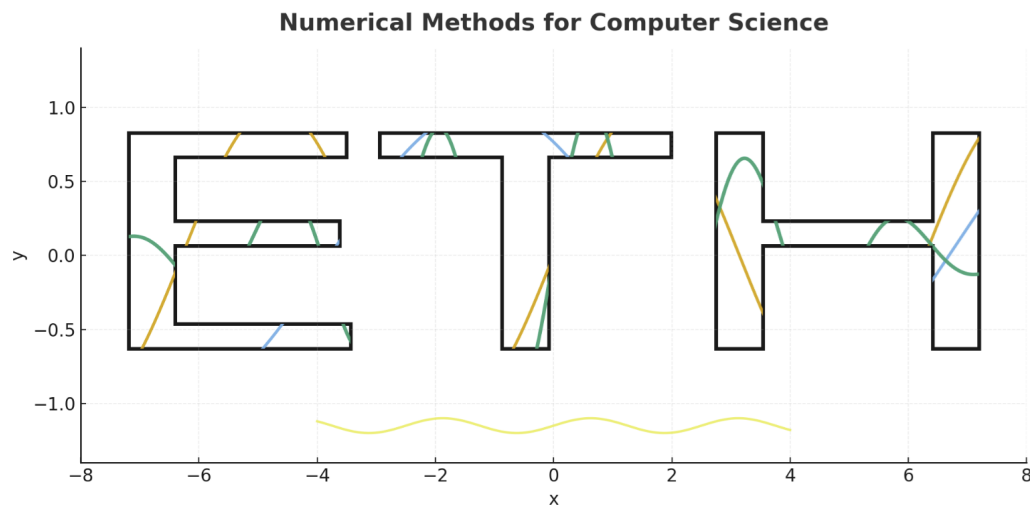




# Iterative Methods for Nonlinear Systems — Week #8

ETH Zurich

DAVID Mihnea-Ştefan

mihdavid@ethz.ch

This week we move from integration to solving nonlinear problems. We explore how iterative methods approach the roots of equations, their convergence properties, and how stopping criteria balance precision and efficiency. Topics include fixed-point iterations, bisection, and Newton's method, from scalar to multi-point approaches.

# Contents

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Motivation and Overview</b>                                      | <b>1</b>  |
| 1.1      | The Challenge of Nonlinear Equations . . . . .                      | 1         |
| 1.2      | Why Iterative (Not Direct) Methods Are Needed . . . . .             | 2         |
| <b>2</b> | <b>Fundamental Concepts</b>                                         | <b>5</b>  |
| 2.1      | What Defines a Nonlinear Equation/System . . . . .                  | 5         |
| 2.2      | Iterative vs. Direct Methods . . . . .                              | 6         |
| 2.3      | Fixed Points and Their Significance . . . . .                       | 7         |
| <b>3</b> | <b>Convergence of Iterative Methods</b>                             | <b>11</b> |
| 3.1      | Definition of Convergence . . . . .                                 | 11        |
| 3.2      | Types of Convergence . . . . .                                      | 11        |
| 3.3      | A Posteriori Error Estimation for Linear Convergence . . . . .      | 12        |
| <b>4</b> | <b>Stopping Criteria and Termination Rules</b>                      | <b>14</b> |
| 4.1      | Why We Cannot Iterate Forever . . . . .                             | 14        |
| 4.2      | Absolute and Relative Error Criteria . . . . .                      | 15        |
| 4.3      | Residual-Based Stopping . . . . .                                   | 15        |
| 4.4      | Machine Precision and Practical Tolerances . . . . .                | 16        |
| <b>5</b> | <b>Bisection Method</b>                                             | <b>17</b> |
| 5.1      | Bracketing the Root: Sign Change Criterion . . . . .                | 17        |
| 5.2      | Step-by-Step Iteration . . . . .                                    | 17        |
| 5.3      | Guaranteed Convergence . . . . .                                    | 18        |
| 5.4      | Trade-Off: Robustness vs. Speed . . . . .                           | 18        |
| <b>6</b> | <b>Fixed-Point Iteration</b>                                        | <b>19</b> |
| 6.1      | From the Idea to an Algorithm . . . . .                             | 19        |
| 6.2      | Geometric Interpretation and Stability . . . . .                    | 19        |
| 6.3      | Example: Different Fixed-Point Forms, Different Behaviors . . . . . | 21        |
| 6.4      | Theoretical Guarantees of Convergence . . . . .                     | 22        |
| <b>7</b> | <b>Newton's Method (Scalar Case)</b>                                | <b>25</b> |
| 7.1      | Idea and Derivation . . . . .                                       | 25        |
| 7.2      | Geometric Interpretation . . . . .                                  | 25        |
| 7.3      | Connection to Fixed-Point Iteration . . . . .                       | 26        |
| 7.4      | Practical Limitations . . . . .                                     | 26        |
| 7.5      | Example . . . . .                                                   | 27        |
| 7.6      | Sensitivity to the Initial Guess . . . . .                          | 27        |
| <b>8</b> | <b>Multi-Point and Higher-Order Methods</b>                         | <b>29</b> |
| 8.1      | Motivation . . . . .                                                | 29        |
| 8.2      | The Secant Method . . . . .                                         | 29        |
| 8.3      | Beyond Two Points - Optional . . . . .                              | 31        |

# 1 Motivation and Overview

Nonlinear equations are everywhere in science and engineering. Unlike linear systems, they rarely admit closed-form solutions and often behave in ways that defy simple algebra. This week introduces *iterative methods*, algorithms that approach the solution step by step, using only evaluations of  $f$  (and sometimes  $f'$ ). The goal is to build practical intuition: when do these methods work, how fast, and how do we stop safely?

**Linear vs. Nonlinear: a first glance.** To appreciate why we need iterative approaches, let us compare two simple systems.

**Linear system (closed form).**

$$\begin{cases} 2x + y = 5, \\ x - y = 1. \end{cases}$$

This can be solved directly using matrix algebra:

$$\begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}, \quad \Rightarrow \quad \mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

*Linear systems always have explicit, algebraic solutions (via inversion or elimination).*

**Nonlinear system (no closed form).**

$$\begin{cases} x^2 + y^2 = 4, \\ e^x + y = 1. \end{cases}$$

Here, the unknowns appear inside nonlinear functions (squares and exponentials), and there is *no algebraic formula* for  $(x, y)$ . We must rely on **iterative methods** to approximate the intersection numerically.

## Note

Linear problems can be “solved once and for all” by Gaussian elimination. Nonlinear problems must be *approximated progressively*: we start with an initial guess, improve it step by step, and stop when changes become negligible.

## 1.1 The Challenge of Nonlinear Equations

A *nonlinear root-finding problem* asks for  $x^*$  such that

$$f(x^*) = 0.$$

Even in one dimension, closed forms are the exception, not the rule:

- **Transcendental equations** (e.g.,  $x = \cos x$ ,  $e^{-x} = x$ ) have no algebraic solutions.
- **High-degree polynomials:** beyond degree 4, Abel–Ruffini tells us that no general formula by radicals exists.
- **Systems  $\mathbf{F}(\mathbf{x}) = \mathbf{0}$**  introduce coupling between variables and can have several solutions or none, depending on parameters.

## Note

Nonlinearity breaks the linear guarantees: superposition fails, there is no universal closed form, and small changes in input can move the root a lot. Numerically, we trade exact formulas for *controlled approximation*, driving the error to zero with a reliable, well-designed iteration.

**Tiny visual thought experiment (1D).** Let  $f(x) = \cos x - x$ . On  $[0, 1]$ ,  $f(0) = 1 > 0$  and  $f(1) \approx -0.459 < 0$ , so a root must lie in between (Intermediate Value Theorem). Yet there is no closed form. This is the canonical situation where iterative methods shine.

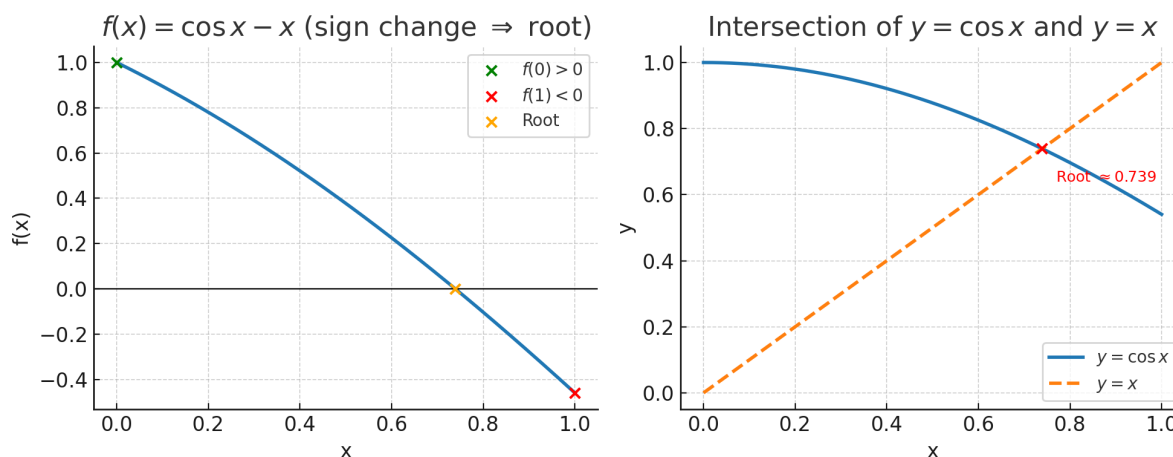


Figure: (Left) Intermediate Value Theorem:  $f(0) > 0$  and  $f(1) < 0$  imply a root in  $(0, 1)$ .  
(Right) The same root is seen as the intersection of  $y = \cos x$  and  $y = x$ .

## 1.2 Why Iterative (Not Direct) Methods Are Needed

In algebra class we learned to solve equations by “moving terms to the other side.” That works for  $2x + 3 = 7$ , but try it on  $x = \cos x$  or  $e^{-x} = x$ , and you quickly realize: there is no formula to rearrange your way out. Most nonlinear problems simply refuse to yield closed forms.

## Note

**Iterative methods** turn this obstacle into a strength. Instead of solving the equation in one magical step, we let the computer *walk toward the answer*, improving its guess at each round. Each iteration brings us closer and we can decide when to stop.

Why we iterate instead of solve directly.

- **Because formulas end.** Beyond degree 4, no general algebraic solution exists (Abel–Ruffini theorem).
- **Because functions get messy.** Exponentials, trigs, and black-box models have no

symbolic inverses.

- **Because we only need an answer, not a formula.** A good numerical root up to  $10^{-8}$  accuracy is worth more than a 3-page symbolic expression.
- **Because we can control the journey.** Iterative solvers can stop early, adapt step sizes, or switch methods when things slow down.
- **Because it's scalable.** The same logic extends naturally to systems  $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ , where each variable depends on all the others, situations far beyond pen-and-paper algebra.

**Think of it like this.** A direct method is like solving a riddle in one brilliant leap. An iterative method is like zooming in with a microscope: each turn of the focus knob reveals a sharper view. Sometimes, patience and feedback beat closed-form genius.

A quick comparison:

|               | Direct (closed form)      | Iterative (numerical)        |
|---------------|---------------------------|------------------------------|
| Availability  | Only in special cases     | Broad, problem-agnostic      |
| Inputs needed | Coefficients/structure    | $f(x)$ , optionally $f'(x)$  |
| Robustness    | Sensitive to conditioning | Can bracket/monitor progress |
| Scalability   | Hard beyond low degree    | Natural for large systems    |
| Stop rule     | N/A                       | User-controlled tolerance    |

**Two minute demos (intuition over algebra).**

- **Bisection (robust):** If  $f(a) \cdot f(b) < 0$ , repeatedly halve the interval. The bracket never fails; the price is a steady (but slow) linear convergence.
- **Fixed-point iteration (simple):** Rewrite  $x = g(x)$  and iterate  $x_{k+1} = g(x_k)$ . If  $|g'(x^*)| < 1$  near the solution, errors shrink like a geometric series.
- **Newton's method (fast near the root):**  $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ . Tangents “jump” to the root with *quadratic* speed once you are close enough.

#### Note

There is no single best method. Bisection is ironclad but slow; Newton is blazing fast but needs a decent initial guess and a nonzero derivative; fixed-point is flexible but must be designed so that  $|g'| < 1$  near the root. A good practitioner mixes robustness (bracketing) with speed (Newton/secant).

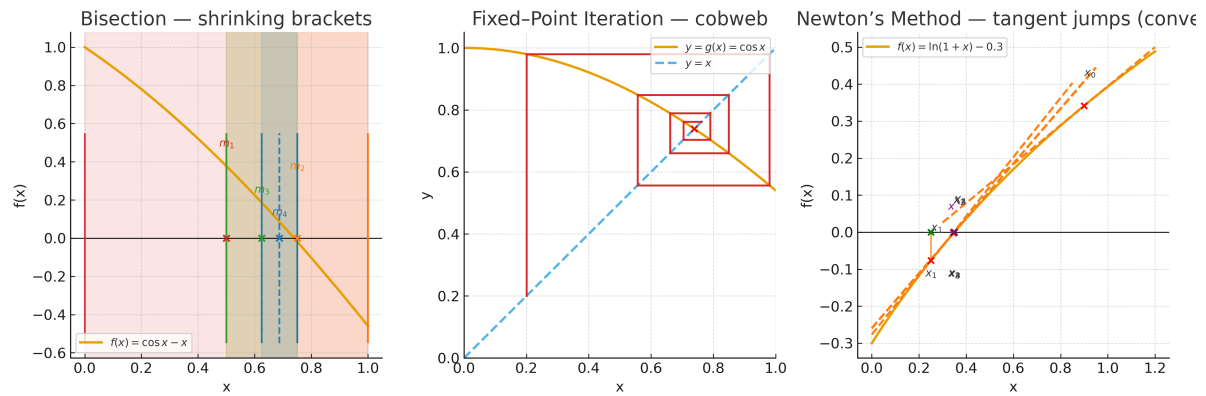


Figure: Three classic solvers with multiple steps. (Left) Bisection halves the bracket each iteration. (Middle) Fixed-point cobweb for  $x = g(x) = \cos x$  converges where  $y = g(x)$  meets  $y = x$ . (Right) Newton's method on a convex function  $f(x) = \ln(1+x) - 0.3$  shows tangent jumps toward the root  $x^*$ .

## 2 Fundamental Concepts

Iterative methods form the backbone of nonlinear numerical analysis. Unlike linear solvers, which can often produce an exact answer in a finite number of steps, nonlinear equations usually demand *progressive approximation*. This section introduces the essential concepts: what defines nonlinearity, what makes a method iterative, and how fixed points serve as the foundation for convergence.

### 2.1 What Defines a Nonlinear Equation/System

A **nonlinear equation** is any equation where the unknown appears inside a nonlinear operation:

$$F(x) = 0, \quad \text{with } F: \mathbb{R}^n \rightarrow \mathbb{R}^n$$

and at least one component of  $F$  is not a linear combination of the variables.

Typical nonlinearities include:

- products or powers of unknowns, e.g.  $x_1x_2$ ,  $x^2$ ,  $\sin(x)$ ,  $e^x$ ;
- coefficients that depend on the solution itself;
- implicit or transcendental relationships.

**Example (Scalar vs. Vector Nonlinearity).**

**Scalar case:**

$$f(x) = e^{-x} - x = 0.$$

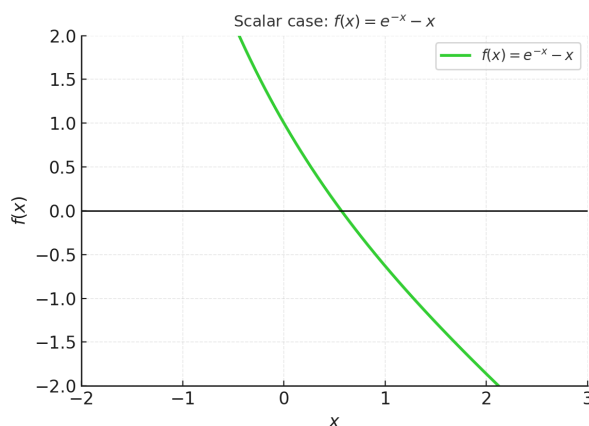
The function mixes exponential and linear terms, no algebraic solution exists.

**Vector case:**

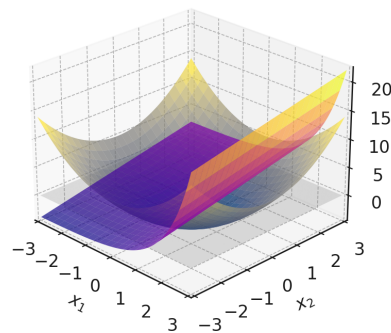
$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} x_1^2 + x_2^2 - 4 \\ e^{x_1} + x_2 - 1 \end{bmatrix} = \mathbf{0}.$$

This couples two nonlinear equations; the solution must be found numerically.

**Nonlinear Behavior: Scalar (2D) vs. Vector (3D) Examples**



**Vector case:  $z = x_1^2 + x_2^2 - 4$  and  $z = e^{x_1} + x_2 - 1$**



**Figure:**

Nonlinear scalar (left) and vector (right) examples. Both involve nonlinear dependencies, yet the vector case couples variables, leading to richer intersection geometry in  $\mathbb{R}^3$ .

## 2.2 Iterative vs. Direct Methods

A **direct method** attempts to solve an equation in finitely many exact arithmetic steps. Gaussian elimination or matrix inversion are typical examples for linear problems.

By contrast, an **iterative method** builds a sequence of approximations:

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots \longrightarrow x^*,$$

where each iterate is computed from previous ones by an *iteration function*:

$$x^{(k+1)} = \Phi(x^{(k)}),$$

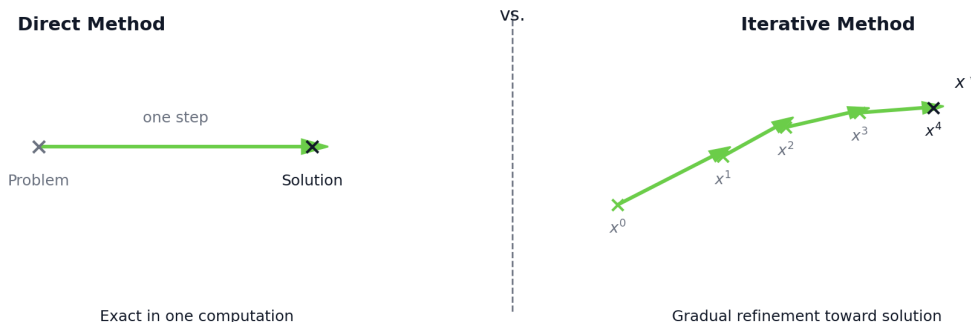
or more generally, for  $m$ -point methods:

$$x^{(k+1)} = \Phi_F(x^{(k)}, x^{(k-1)}, \dots, x^{(k-m+1)}).$$

The first  $m$  elements  $x^{(0)}, x^{(1)}, \dots, x^{(m-1)}$  are called **initial guesses** or **guess points**, chosen by the user at the start of the algorithm. They serve as the entry point for the iteration process and can be selected based on prior knowledge, physical intuition, or even at random. However, determining good initial guesses is often nontrivial: a poor choice can slow convergence or even cause divergence, especially for nonlinear problems with multiple roots.

### Note

The function  $\Phi_F$  defines the entire algorithm. It maps current estimates into new ones, and its fixed points correspond to the solutions of the original system  $F(x) = 0$ . The performance and stability of the method depend not only on  $\Phi_F$  but also on how well the initial  $m$  guesses approximate the true root.



Direct vs. Iterative solution. A direct method computes the answer in one step, while an iterative method refines successive approximations until reaching the target solution  $x^*$ .



## 2.3 Fixed Points and Their Significance

A **fixed point** of a mapping  $\Phi : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^n$  is a vector  $x^*$  such that

$$\Phi(x^*) = x^*.$$

Intuitively, applying  $\Phi$  to  $x^*$  leaves it unchanged: the iteration “stops moving.” In numerical methods, this property encodes the very definition of having *found the solution* (because we can’t reduce the error anymore).

Suppose our original nonlinear problem is

$$F(x) = 0.$$

If we can rewrite it in an equivalent form

$$x = \Phi(x),$$

then the fixed point  $x^*$  of  $\Phi$  automatically satisfies  $F(x^*) = 0$ . This connection is the cornerstone of all iterative methods: finding a root of  $F$  is the same as finding a fixed point of  $\Phi$ .

### Note

A well-designed iteration must be *consistent*:

$$\Phi(x^*) = x^* \quad \Leftrightarrow \quad F(x^*) = 0.$$

Otherwise, the fixed point of  $\Phi$  might not correspond to a true root of  $F$ . Consistency ensures that the algorithm’s stationary points are the correct mathematical solutions.

**Example: From root finding to fixed point.** Consider the nonlinear equation

$$F(x) = \cos x - x = 0.$$

This is one of the most classical transcendental equations, its root cannot be expressed in closed form.

We can rewrite it as an equivalent fixed-point problem:

$$x = \cos x,$$

which means that any solution  $x^*$  where the cosine curve intersects the line  $y = x$  will automatically satisfy  $F(x^*) = 0$ .

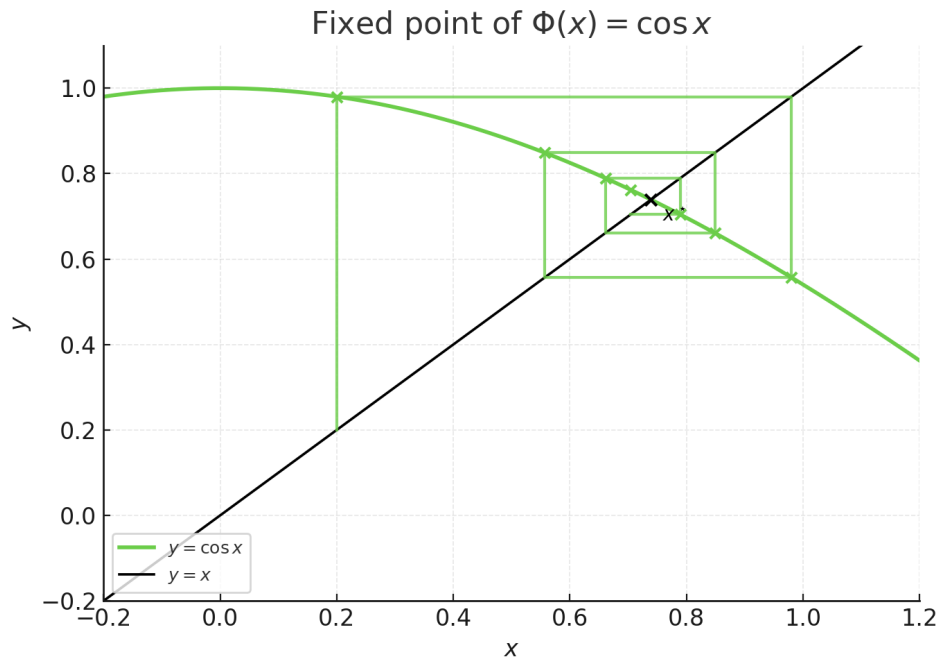


Figure: The root of  $F(x) = \cos x - x$  is the intersection between  $y = \cos x$  and  $y = x$ . The same point  $x^*$  is the fixed point of the mapping  $\Phi(x) = \cos x$ .

#### Note

At the fixed point  $x^*$ , applying the iteration  $\Phi(x) = \cos x$  leaves the value unchanged:  $\Phi(x^*) = x^*$ . For nearby values, repeated iteration  $x_{k+1} = \cos x_k$  moves the points closer to this intersection, as long as  $|\Phi'(x^*)| < 1$ . Thus, the numerical process of “solving  $F(x) = 0$ ” becomes the geometric process of “finding where  $y = \Phi(x)$  meets  $y = x$ .”

#### Note

The “staircase” or “spiral” path in the figure illustrates the logic of fixed-point iteration. Starting from an initial guess  $x^{(0)}$ , we move *vertically* to the curve  $y = \Phi(x)$ , which represents evaluating  $\Phi(x^{(0)})$ . Then, we move *horizontally* to the diagonal  $y = x$ , so that the  $x$ -coordinate of this new point becomes the next iterate  $x^{(1)}$ .

Geometrically, this horizontal step is like “projecting back” onto the axis of input values: it ensures that the next iteration starts from the value just computed, not from the height of the curve. Each pair of vertical–horizontal moves represents one update  $x^{(k+1)} = \Phi(x^{(k)})$ , and if  $|\Phi'(x^*)| < 1$ , these steps contract toward the fixed point  $x^*$ .

In other words, when you “move right until you hit  $y = x$ ,” you are aligning your output  $\Phi(x^{(k)})$  with the next input, exactly the value you seek to refine. This is what makes the cobweb diagram a faithful visual metaphor for the iteration process.

**Why this equivalence is true.** If  $x^*$  satisfies  $F(x^*) = 0$ , then any function  $\Phi$  of the form

$$\Phi(x) = x - B(x)F(x),$$

with some matrix-valued operator  $B(x)$  (for example  $B(x) = J_F(x)^{-1}$  in Newton's method), will have  $\Phi(x^*) = x^*$  automatically, since  $F(x^*) = 0$  makes the correction term vanish. Conversely, if  $\Phi(x^*) = x^*$ , then substituting back gives

$$0 = \Phi(x^*) - x^* = -B(x^*)F(x^*),$$

and provided  $B(x^*)$  is nonsingular, this implies  $F(x^*) = 0$ . Thus, the two problems are algebraically equivalent.

#### Note

This equivalence is what allows us to replace “solve  $F(x) = 0$ ” by “find a fixed point of  $\Phi$ ” without losing correctness. The success of the iteration then depends on whether  $\Phi$  pulls points toward  $x^*$  (convergent) or pushes them away (divergent).

**Geometric and conceptual meaning.** Visually, a fixed point is the intersection of the curve  $y = \Phi(x)$  with the diagonal  $y = x$ . Each iteration moves from  $x_k$  vertically to  $y = \Phi(x)$  (evaluation step), then horizontally back to  $y = x$  (projection step). Repeating this motion generates the familiar *cobweb diagram*, whose pattern directly reveals the behavior of the iteration.

The key factor governing convergence is the slope of  $\Phi$  at the fixed point:

$$|\Phi'(x^*)| < 1 \quad \Rightarrow \quad \text{local convergence.}$$

Intuitively, this means that  $\Phi$  is “flat enough” near the intersection, nearby points are pulled inward rather than pushed away.

- If  $|\Phi'(x^*)| < 1$ , the staircase contracts toward  $x^*$ .
- If  $|\Phi'(x^*)| > 1$ , the staircase expands and diverges.
- If  $|\Phi'(x^*)| = 1$ , we are in a neutral or oscillatory situation: convergence is not guaranteed and small perturbations can lead to cycles.

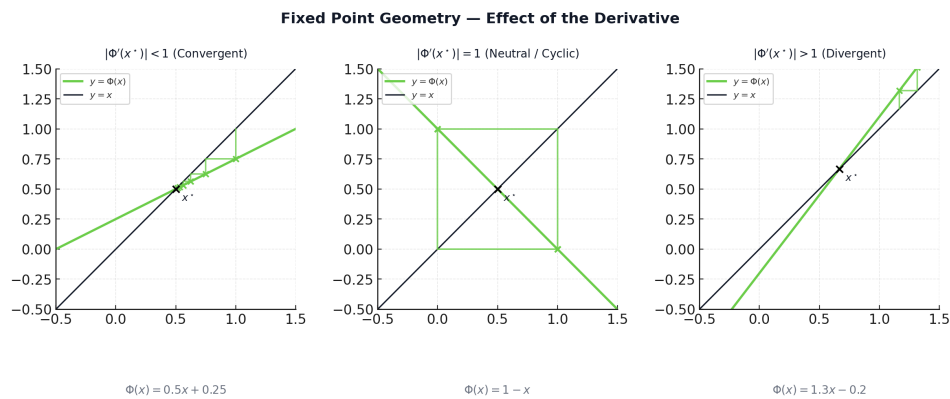


Figure: Fixed point as intersection of  $y = x$  and  $y = \Phi(x)$  (1D case). The slope  $|\Phi'(x^*)|$  determines whether iterates spiral inward (stable) or outward (unstable).

**Example: why the cosine map is safe.** For  $\Phi(x) = \cos x$ , we have  $\Phi'(x) = -\sin x$ . Since  $|\sin x| \leq 1$  for all real  $x$ , we immediately know that

$$|\Phi'(x)| = |\sin x| < 1$$

except at isolated points where  $|\sin x| = 1$  (which do not correspond to the fixed point). Therefore, even without knowing the exact value of  $x^*$ , we can be sure that the iteration  $x_{k+1} = \cos x_k$  is locally convergent.

#### Note

This is a powerful idea: sometimes we can guarantee convergence *without computing the solution itself*, simply by bounding the derivative. The derivative acts as a “stability detector”, its magnitude tells us whether the iteration pulls errors inward or amplifies them.

**Note that we still didn’t say how to find a good iteration function**

### 3 Convergence of Iterative Methods

Once an iterative scheme has been defined, the next question is not *if* it converges, but *how fast*. Convergence analysis provides the mathematical tools to measure and compare the efficiency of different algorithms.

#### 3.1 Definition of Convergence

Let  $\{x^{(k)}\}_{k=0}^{\infty}$  be the sequence generated by an iteration. We say that the method **converges** to  $x^*$  if

$$\lim_{k \rightarrow \infty} x^{(k)} = x^*.$$

The difference between the current iterate and the true solution is the **error**:

$$e^{(k)} = x^{(k)} - x^*.$$

#### Note

Convergence alone only guarantees that  $x^{(k)}$  gets closer to  $x^*$  as  $k$  increases, but says nothing about the *speed* of that approach. Different methods can converge to the same solution with vastly different rates.

#### 3.2 Types of Convergence

To characterize how quickly the sequence  $\{x^{(k)}\}$  approaches  $x^*$ , we compare successive errors. A method is said to have **order of convergence**  $p > 0$  if

$$\lim_{k \rightarrow \infty} \frac{|e^{(k+1)}|}{|e^{(k)}|^p} = C,$$

for some constant  $C > 0$ , called the **asymptotic error constant**. Intuitively:

- $p = 1$  — **linear convergence**: error shrinks by a constant factor at each step.
- $p > 1$  — **superlinear convergence**: error decreases faster than linearly.
- $p = 2$  — **quadratic convergence**: the number of correct digits roughly doubles each iteration (we will see this in practice soon).

For example, if  $|e^{(k+1)}| \approx 0.1 |e^{(k)}|$ , then  $p = 1$  and  $C = 0.1$  (slow, steady progress). If instead  $|e^{(k+1)}| \approx (|e^{(k)}|)^2$ , then  $p = 2$  and convergence accelerates dramatically near the root.

## Note

Linear convergence means the iteration behaves like a geometric series:

$$|e^{(k)}| \approx C^k |e^{(0)}|, \quad 0 < C < 1.$$

Quadratic convergence means that once  $e^{(k)}$  becomes small, the next error is roughly proportional to its square:

$$|e^{(k+1)}| \approx C |e^{(k)}|^2.$$

Such methods “explode in precision” near the solution, typical of Newton’s method and its variants.

### 3.3 A Posteriori Error Estimation for Linear Convergence

In theory, convergence is described in terms of the true error  $e^{(k)} = x^{(k)} - x^*$ . However, in numerical practice we never know  $x^*$  in advance, the whole point of the iteration is to approximate it. Therefore, a key question is: *how can we estimate the error without knowing the exact solution?*

**From definition to practical bound.** For a linearly convergent sequence with rate  $0 < L < 1$ , we know that

$$\|x^{(k+1)} - x^*\| \leq L \|x^{(k)} - x^*\|, \quad \forall k.$$

To eliminate the unknown  $x^*$ , we manipulate this inequality using the triangle inequality:

$$\|x^{(k)} - x^*\| \leq \|x^{(k+1)} - x^{(k)}\| + \|x^{(k+1)} - x^*\|.$$

Substituting this into the original relation gives

$$\|x^{(k+1)} - x^*\| \leq L(\|x^{(k+1)} - x^{(k)}\| + \|x^{(k+1)} - x^*\|).$$

After rearranging terms, we obtain the **a posteriori error estimate**:

$$\|x^{(k+1)} - x^*\| \leq \frac{L}{1-L} \|x^{(k+1)} - x^{(k)}\|.$$

The derivation is purely algebraic but extremely powerful: it replaces the inaccessible true error  $\|x^{(k)} - x^*\|$  with a computable quantity  $\|x^{(k+1)} - x^{(k)}\|$  that involves only successive iterates.

**Practical interpretation.** This inequality shows that, under linear convergence, the distance between successive iterates acts as a reliable proxy for the true error. If the difference  $\|x^{(k+1)} - x^{(k)}\|$  becomes very small, we can guarantee that  $\|x^{(k+1)} - x^*\|$  is smaller still, at most  $\frac{L}{1-L}$  times that quantity.

In numerical codes, this relation provides a **safe and verifiable stopping rule**: we can stop the iteration once

$$\|x^{(k+1)} - x^{(k)}\| < \text{tolerance}.$$

**Optional reference: verifying the bound in practice.** A similar experiment appears in the lecture notes **NumCS(E)**, Prof. R. Hiptmair, SAM, ETH Zurich, 2025. It revisits the iteration

$$x^{(k+1)} = x^{(k)} + \frac{\cos x^{(k)} + 1}{\sin x^{(k)}} \Rightarrow x^{(k)} \rightarrow \pi,$$

which exhibits a linear convergence rate of  $L = \frac{1}{2}$  for  $x^{(0)}$  near  $\pi$ . The following table (adapted from the lecture notes) illustrates the *a posteriori* error bound:

$$|x^{(k+1)} - x^*| \leq \frac{L}{1-L} |x^{(k+1)} - x^{(k)}|.$$

Observed rate of convergence:  $L = 1/2$

Error and error bound for  $x^{(0)} = 0.4$ :

| $k$ | $ x^{(k)} - \pi $ | $\frac{L}{1-L}  x^{(k)} - x^{(k-1)} $ | slack of bound    |
|-----|-------------------|---------------------------------------|-------------------|
| 1   | 2.191562221997101 | 4.933154875586894                     | 2.741592653589793 |
| 2   | 0.247139097781070 | 1.944423124216031                     | 1.697284026434961 |
| 3   | 0.122936737876834 | 0.124202359904236                     | 0.001265622027401 |
| 4   | 0.061390835206217 | 0.061545902670618                     | 0.000155067464401 |
| 5   | 0.030685773472263 | 0.030705061733954                     | 0.000019288261691 |
| 6   | 0.015341682696235 | 0.015344090776028                     | 0.000002408079792 |
| 7   | 0.007670690889185 | 0.007670991807050                     | 0.000000300917864 |
| 8   | 0.003835326638666 | 0.003835364250520                     | 0.000000037611854 |
| 9   | 0.001917660968637 | 0.001917665670029                     | 0.000000004701392 |
| 10  | 0.000958830190489 | 0.000958830778147                     | 0.000000000587658 |
| 11  | 0.000479415058549 | 0.000479415131941                     | 0.000000000073392 |
| 12  | 0.000239707524646 | 0.000239707533903                     | 0.000000000009257 |
| 13  | 0.000119853761949 | 0.000119853762696                     | 0.000000000000747 |
| 14  | 0.000059926881308 | 0.000059926880641                     | 0.000000000000667 |
| 15  | 0.000029963440745 | 0.000029963440563                     | 0.000000000000181 |

Figure: *A posteriori* error bound for linearly convergent iteration. Source: **NumCS(E) Lecture Notes 2025** — Prof. R. Hiptmair, SAM, ETH Zurich.

#### Note

The results confirm that the bound  $\frac{L}{1-L} |x^{(k+1)} - x^{(k)}|$  is highly accurate: the difference (*slack*) between the estimated and the true error quickly becomes negligible. This justifies using  $\|x^{(k+1)} - x^{(k)}\|$  as a safe and practical stopping indicator in numerical implementations.

## 4 Stopping Criteria and Termination Rules

Iterative methods do not produce the exact solution in a finite number of steps. Instead, they *approach* it progressively. The key practical question is *when to stop*. Stopping too early leaves a large residual; stopping too late wastes computation and may even amplify roundoff errors.

### Note

In numerical computing, we never reach the exact root  $x^*$ , but we can guarantee that our current iterate  $x^{(k)}$  is *close enough* according to a well-chosen tolerance.

### 4.1 Why We Cannot Iterate Forever

Even if convergence is guaranteed, performing too many iterations can:

- waste computational time when progress becomes negligible,
- amplify floating-point roundoff errors,
- or cause overflow/underflow in ill-conditioned problems.

Each iteration consumes time, memory, and numerical precision. Hence, practical solvers always include a **termination condition** to decide when the iteration has achieved sufficient accuracy.

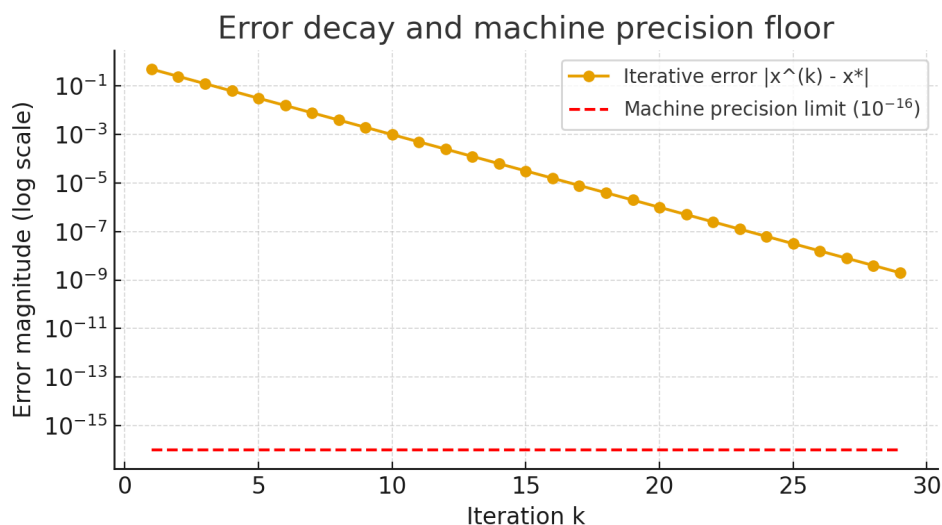


Figure: Iterative error decreases until it hits the *machine precision floor*. Past this point, numerical noise dominates, further iterations are pointless.

**Diminishing returns: why more iterations do not always help.** Even for a method with *quadratic convergence*, the improvement per step quickly becomes insignificant once the error is small. In the previous example, the error decreases from  $10^{-1}$  to  $10^{-6}$  in only four steps. Beyond that point, numerical roundoff and finite precision dominate.



| $k$ | $x^{(k)}$   | $ e^{(k)}  =  x^{(k)} - \sqrt{2} $        |
|-----|-------------|-------------------------------------------|
| 0   | 1.000000000 | $4.1 \times 10^{-1}$                      |
| 1   | 1.500000000 | $8.6 \times 10^{-2}$                      |
| 2   | 1.416666667 | $2.4 \times 10^{-3}$                      |
| 3   | 1.414215686 | $2.1 \times 10^{-6}$                      |
| 4   | 1.414213562 | $2.2 \times 10^{-16}$ (machine precision) |

By iteration 4, the error has reached the limits of double-precision arithmetic. Continuing beyond this point will not yield a more accurate result, only more computation and accumulated floating-point noise.

#### Note

This is the practical motivation behind **stopping criteria**: once the error (or the change between iterates) falls below a given threshold, the iteration should terminate. Beyond that, the gain in accuracy is illusory, we simply chase numerical noise.

## 4.2 Absolute and Relative Error Criteria

There are two fundamental ways to measure “how close” we are to the true solution.

### Absolute error.

$$\|x^{(k+1)} - x^{(k)}\| < \varepsilon_{\text{abs}}$$

This criterion stops when consecutive iterates differ by less than a fixed tolerance  $\varepsilon_{\text{abs}}$ .

### Relative error.

$$\frac{\|x^{(k+1)} - x^{(k)}\|}{\|x^{(k+1)}\|} < \varepsilon_{\text{rel}}$$

This scales the threshold by the magnitude of the current iterate, ensuring that large and small solutions are treated consistently.

#### Note

Absolute errors are useful when the scale of the problem is known in advance (e.g. we expect  $x^* \approx 1$ ). Relative errors are preferred when variables can vary by orders of magnitude. In practice, most numerical codes combine both:

$$\|x^{(k+1)} - x^{(k)}\| < \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \|x^{(k+1)}\|.$$

## 4.3 Residual-Based Stopping

Alternatively, one may monitor the **residual**:

$$r^{(k)} = F(x^{(k)}), \quad \|r^{(k)}\| = \|F(x^{(k)})\| < \varepsilon_{\text{res}}.$$

This approach is intuitive, since the root itself satisfies  $F(x^*) = 0$ .

- Small residual  $\Rightarrow$  the function value is close to zero, so we *may* be near a root.
- However, beware:
  - a small residual does **not always** imply a small error if the Jacobian  $J_F(x)$  is ill-conditioned;
  - sometimes  $F(x)$  has a **shallow local minimum** near zero, but never crosses the axis.

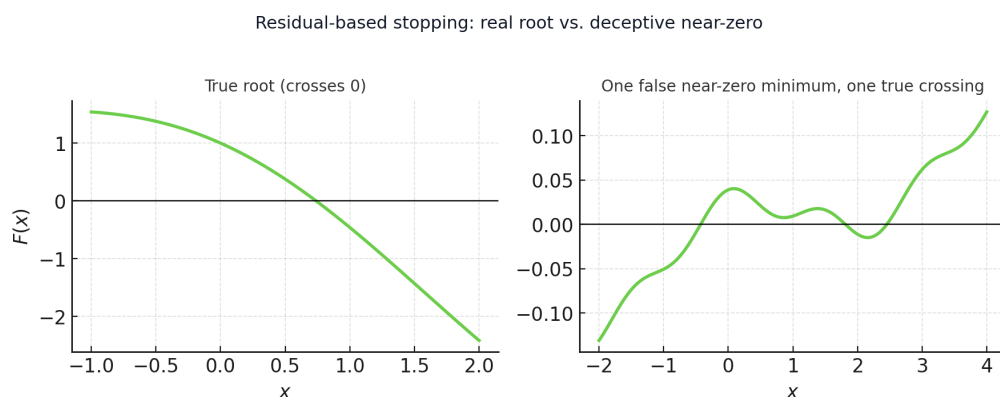


Figure: (Left) a genuine root where  $F(x)$  crosses 0. (Right) one false minimum near zero and one true crossing. Depending on the starting guess, an iterative solver may converge to the wrong (false) stationary point.

#### Note

Residual-based stopping is powerful but potentially misleading: it confirms that the function value is small, not that the correct root has been found. For functions with multiple local minima, this becomes a major issue, especially for Newton's method, which can converge to whichever minimum is *closest to the initial guess*.

## 4.4 Machine Precision and Practical Tolerances

Floating-point arithmetic limits achievable accuracy. For double precision (`float64`), the smallest distinguishable difference is

$$\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}.$$

Iterating beyond this threshold no longer improves accuracy, differences smaller than  $\varepsilon_{\text{mach}}$  are invisible to the computer.

Typical tolerances in engineering and scientific computing:

$$\varepsilon_{\text{abs}} = 10^{-8}, \quad \varepsilon_{\text{rel}} = 10^{-6}.$$

## 5 Bisection Method

The **Bisection Method** is the simplest and most reliable root-finding technique. It applies only to continuous scalar functions, but its key advantage is *guaranteed convergence*.

It embodies the idea of *controlled iteration*: each step halves the uncertainty interval while ensuring the root remains enclosed.

### 5.1 Bracketing the Root: Sign Change Criterion

Let  $f : [a, b] \rightarrow \mathbb{R}$  be continuous and suppose

$$f(a)f(b) < 0.$$

By the Intermediate Value Theorem, there exists at least one root  $x^* \in (a, b)$  such that  $f(x^*) = 0$ .

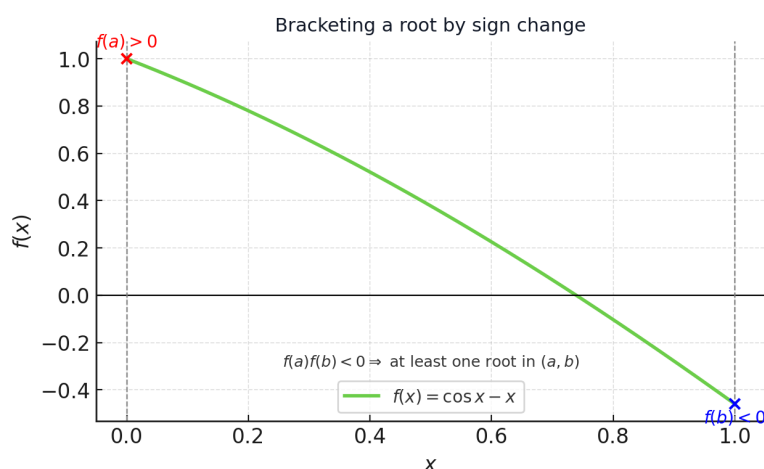


Figure: Since  $f(a)$  and  $f(b)$  have opposite signs, a zero must lie in between.

### 5.2 Step-by-Step Iteration

At each iteration  $k$ :

$$m_k = \frac{a_k + b_k}{2}, \quad f_m = f(m_k).$$

Depending on the sign of  $f(a_k)f_m$ :

$$\begin{cases} b_{k+1} = m_k, & \text{if } f(a_k)f_m < 0, \\ a_{k+1} = m_k, & \text{else.} \end{cases}$$

Thus the interval length shrinks geometrically:  $|b_k - a_k| = \frac{b_0 - a_0}{2^k}$ .

#### Note

After each step, the new interval still satisfies the sign-change condition. The method never “loses” the root, this is what makes it robust.

### 5.3 Guaranteed Convergence

Because the bracketing interval halves every iteration, the midpoint  $m_k$  approaches the true root  $x^*$  linearly:

$$|m_k - x^*| \leq \frac{b_0 - a_0}{2^{k+1}}.$$

Hence, to achieve an absolute accuracy  $\varepsilon$ :

$$k \geq \log_2 \left( \frac{b_0 - a_0}{\varepsilon} \right).$$

#### Note

Bisection's convergence is **linear** ( $p = 1$ ), but the rate is predictable and independent of the function shape. No derivative or curvature information is required.

### 5.4 Trade-Off: Robustness vs. Speed

- **Pros:** Always converges if  $f$  is continuous and  $f(a)f(b) < 0$ . The interval reliably encloses the true root.
- **Cons:** Convergence is only linear; each step halves the error. Compared to higher-order methods (like Newton), it is slow.

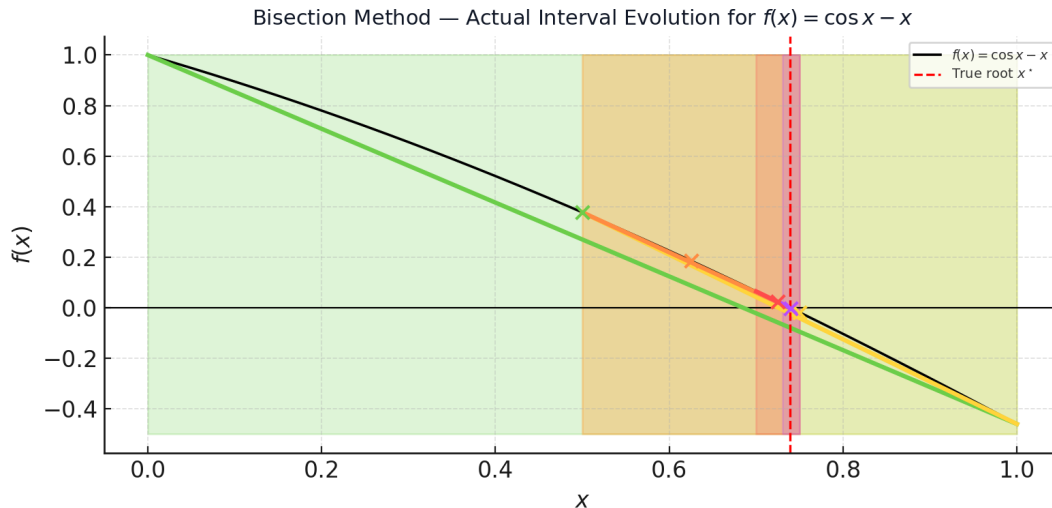


Figure: Bisection in action. At each iteration, the interval  $[a_k, b_k]$  shrinks and the midpoint  $m_k$  approaches the true root.

#### Note

Bisection is often used to initialize faster methods: it first narrows down a guaranteed region, then Newton or secant methods take over for rapid refinement.

## 6 Fixed-Point Iteration

In the previous sections, we established that solving a nonlinear equation

$$F(x) = 0$$

is equivalent to finding a **fixed point** of a suitable mapping  $\Phi(x)$ , satisfying

$$x^* = \Phi(x^*).$$

We now turn this geometric and theoretical concept into an explicit *algorithmic procedure*.

### 6.1 From the Idea to an Algorithm

The essential step is to construct a function  $\Phi$  such that its fixed points correspond exactly to the roots of  $F(x)$ . Starting from an initial guess  $x^{(0)}$ , we then generate a sequence:

$$x^{(k+1)} = \Phi(x^{(k)}), \quad k = 0, 1, 2, \dots$$

#### Note

The choice of  $\Phi$  is not unique. Different rearrangements of the same equation  $F(x) = 0$  may lead to entirely different convergence behavior. The fixed-point framework is thus not a single method, but a *family of iterative schemes*.

**Consistency.** To ensure correctness, the mapping must be consistent:

$$\Phi(x^*) = x^* \iff F(x^*) = 0.$$

Otherwise, an iteration might converge to a point that is not a true root.

### 6.2 Geometric Interpretation and Stability

Recall from our earlier discussion that the equation  $x = \Phi(x)$  can be visualized as the intersection of the curves  $y = \Phi(x)$  and  $y = x$ . The iterative process alternates between these two:

$$x_k \xrightarrow{\text{vertical}} y = \Phi(x_k) \xrightarrow{\text{horizontal}} y = x,$$

producing the classical “cobweb diagram.”

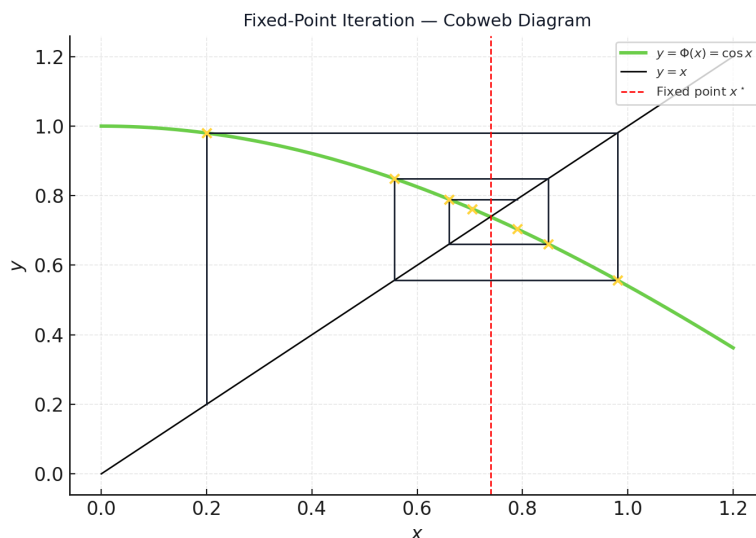


Figure: Graphical view of the fixed-point iteration  $x_{k+1} = \Phi(x_k)$ . Each step moves vertically to  $y = \Phi(x)$ , then horizontally to  $y = x$ .

The geometry immediately reveals the stability criterion: the iteration converges if  $\Phi$  is “flat enough” near the fixed point:

$$|\Phi'(x^*)| < 1.$$

When this holds, the sequence  $\{x^{(k)}\}$  spirals toward  $x^*$ ; otherwise, it diverges or oscillates.

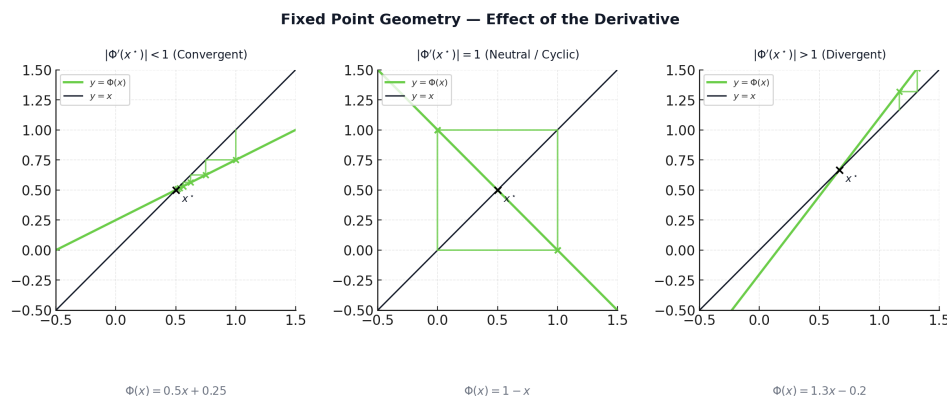


Figure: Fixed point as intersection of  $y = x$  and  $y = \Phi(x)$  (1D case). The slope  $|\Phi'(x^*)|$  determines whether iterates spiral inward (stable) or outward (unstable).

### Note

This criterion comes from the first-order local error model:

$$e_{k+1} = \Phi'(x^*)e_k + \mathcal{O}(e_k^2),$$

so if  $|\Phi'(x^*)| < 1$ , the error decreases approximately by a factor  $|\Phi'(x^*)|$  each step (so at least linear).

### 6.3 Example: Different Fixed-Point Forms, Different Behaviors

Consider the nonlinear equation

$$F(x) = xe^x - 1 = 0,$$

whose true root is approximately  $x^* \approx 0.567143$  (the so-called *Omega constant*).

We can rewrite  $F(x) = 0$  in several equivalent fixed-point forms:

$$\Phi_1(x) = e^{-x}, \quad \Phi_2(x) = \frac{1+x}{1+e^x}, \quad \Phi_3(x) = x + 1 - xe^x.$$

Each formulation defines a different iteration scheme:

$$x_{k+1} = \Phi_i(x_k), \quad i = 1, 2, 3.$$

**Experiment.** Starting from the same initial guess  $x^{(0)} = 0$ , we compute the iterates for each variant up to five steps.

| Iteration                       | $x^{(0)}$ | $x^{(1)}$ | $x^{(2)}$ | $x^{(3)}$ | $x^{(4)}$ | $x^{(5)}$ |
|---------------------------------|-----------|-----------|-----------|-----------|-----------|-----------|
| $\Phi_1(x) = e^{-x}$            | 0.000     | 1.000     | 0.368     | 0.692     | 0.501     | 0.607     |
| $\Phi_2(x) = \frac{1+x}{1+e^x}$ | 0.000     | 0.500     | 0.565     | 0.567     | 0.567     | 0.567     |
| $\Phi_3(x) = x + 1 - xe^x$      | 0.000     | 1.000     | 0.000     | 1.000     | 0.000     | 1.000     |

#### Observations.

- $\Phi_1$ : converges slowly but steadily. Indeed,  $\Phi_1'(x^*) = -e^{-x^*} \approx -0.567$ , so convergence is linear with ratio  $\lambda \approx 0.57$ .
- $\Phi_2$ : converges very quickly. Here,  $\Phi_2'(x^*) \approx 0.003$ , indicating extremely fast linear (almost quadratic) convergence.
- $\Phi_3$ : oscillates forever between 0 and 1 — divergence. Since  $\Phi_3'(x^*) \approx 1.56 > 1$ , the fixed point is unstable.

#### Note

This example illustrates that even though all three  $\Phi_i$  are algebraically equivalent to the same equation  $F(x) = 0$ , only some of them lead to a convergent iteration. The derivative  $|\Phi'(x^*)|$  fully determines the local behavior.

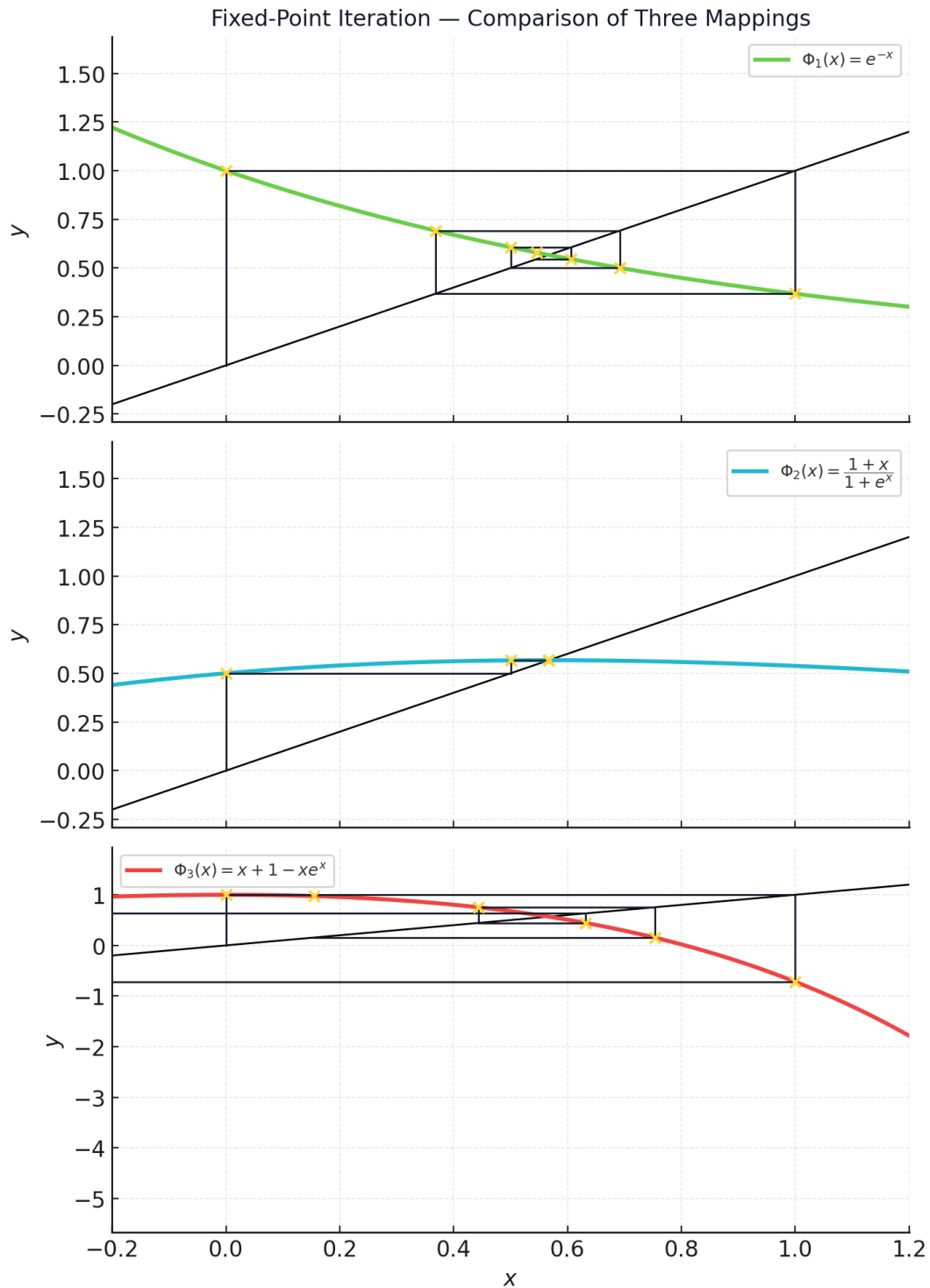


Figure: Three fixed-point functions for  $F(x) = xe^x - 1$ . Only  $\Phi_1$  and  $\Phi_2$  cross  $y = x$  with slope magnitude  $< 1$ .

#### 6.4 Theoretical Guarantees of Convergence

So far, we have reasoned intuitively that if the slope of the iteration function  $\Phi(x)$  near the fixed point is sufficiently small, then the sequence

$$x^{(k+1)} = \Phi(x^{(k)})$$



should converge. We now state the precise mathematical conditions that make this true.

**Local Linear Convergence.** Let  $\Phi : U \subset \mathbb{R} \rightarrow \mathbb{R}$  be continuously differentiable, and suppose  $\Phi(x^*) = x^*$ . If the derivative at the fixed point satisfies

$$|\Phi'(x^*)| < 1,$$

then the iteration converges *locally and at least linearly* to  $x^*$ , that is,

$$\exists L \in (0, 1) : \quad |x^{(k+1)} - x^*| \leq L |x^{(k)} - x^*|, \quad \text{for all } k \text{ sufficiently large.}$$

This criterion follows from the first-order local error model:

$$e_{k+1} = \Phi'(x^*)e_k + \mathcal{O}(e_k^2),$$

so if  $|\Phi'(x^*)| < 1$ , the error decreases approximately by a factor  $|\Phi'(x^*)|$  at each step.

#### Note

In higher dimensions, the condition becomes  $\|D\Phi(x^*)\| < 1$ , where  $D\Phi(x^*)$  is the Jacobian matrix of  $\Phi$ . It means that the iteration acts as a *local contraction* near the fixed point.

**Interpretation.** This result guarantees at least geometric (linear) convergence. However, it does not exclude faster convergence rates. To see why, expand  $\Phi$  in a Taylor series near  $x^*$ :

$$\Phi(x) = \Phi(x^*) + \Phi'(x^*)(x - x^*) + \frac{1}{2}\Phi''(x^*)(x - x^*)^2 + \mathcal{O}((x - x^*)^3).$$

If  $\Phi'(x^*)$  is small or even zero, the higher-order terms dominate, leading to faster-than-linear convergence.

**Higher-Order Convergence.** If  $\Phi$  is  $(m + 1)$ -times continuously differentiable and

$$\Phi^{(l)}(x^*) = 0, \quad \text{for all } l = 1, \dots, m,$$

then the fixed-point iteration converges *locally* to  $x^*$  with order at least  $m + 1$ :

$$|x^{(k+1)} - x^*| \leq C |x^{(k)} - x^*|^{m+1}.$$

**Why “at least” linear?** The term “at least” reflects that linear convergence is only the minimal rate guaranteed by the derivative test. If  $\Phi'(x^*)$  happens to vanish, the iteration immediately becomes superlinear or quadratic.

| Condition near $x^*$                                | Resulting convergence order |
|-----------------------------------------------------|-----------------------------|
| $ \Phi'(x^*)  < 1$                                  | Linear                      |
| $\Phi'(x^*) = 0, \Phi''(x^*) \neq 0$                | Quadratic                   |
| $\Phi'(x^*) = \Phi''(x^*) = 0, \Phi'''(x^*) \neq 0$ | Cubic                       |

#### Note

This theoretical framework unifies all iterative root-finding methods. Newton's and Secant methods can both be expressed as fixed-point iterations with specially designed  $\Phi(x)$  that satisfy  $\Phi'(x^*) = 0$ , hence achieving superlinear or quadratic convergence.

## 7 Newton's Method (Scalar Case)

Among all iterative schemes, **Newton's method** plays a central role. It refines the general fixed-point concept into a powerful root-finding technique with *quadratic convergence* near the solution.

### 7.1 Idea and Derivation

We seek a root of  $f(x) = 0$ . At iteration  $k$ , assume we have an approximation  $x^{(k)}$ . Using a first-order Taylor expansion of  $f$  around  $x^{(k)}$ :

$$f(x) \approx f(x^{(k)}) + f'(x^{(k)})(x - x^{(k)}).$$

Setting this linear model to zero gives an estimate of the root:

$$0 = f(x^{(k)}) + f'(x^{(k)})(x^{(k+1)} - x^{(k)}),$$

from which we obtain the classical Newton update:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}.$$

Each step uses the tangent line of  $f$  at  $x^{(k)}$  to approximate where it crosses the  $x$ -axis. Hence, Newton's method can also be seen as a specific *fixed-point iteration* with

$$\Phi(x) = x - \frac{f(x)}{f'(x)}.$$

### 7.2 Geometric Interpretation

Geometrically, at each step we draw the tangent to  $y = f(x)$  at the current point and move to its intersection with the  $x$ -axis.

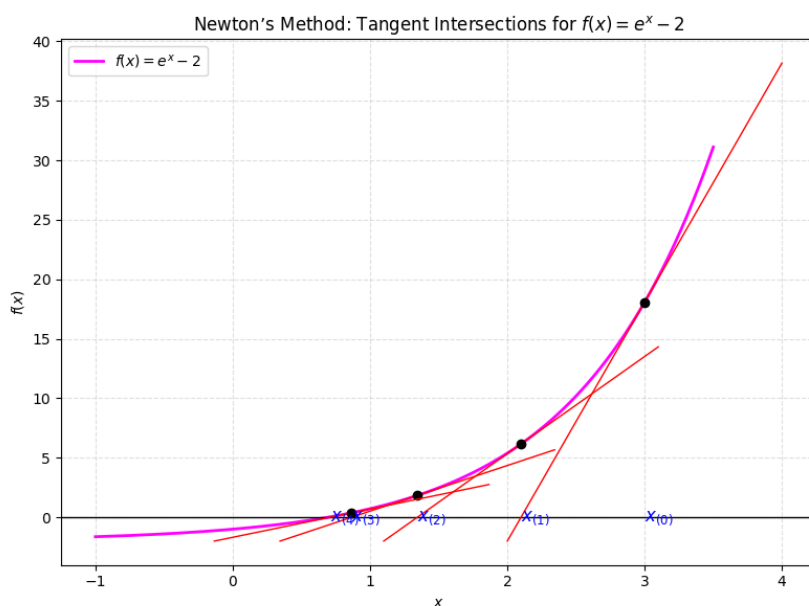


Figure: Newton's method as tangent intersection. The next iterate  $x_{k+1}$  is where the tangent at  $x_k$  crosses the axis.

### 7.3 Connection to Fixed-Point Iteration

Formally, Newton's method is a particular case of the fixed-point scheme:

$$x_{k+1} = \Phi(x_k), \quad \Phi(x) = x - \frac{f(x)}{f'(x)}.$$

To find its derivative, we apply the quotient rule to the term  $\frac{f(x)}{f'(x)}$ :

$$\frac{d}{dx} \left( \frac{f(x)}{f'(x)} \right) = \frac{f'(x) f'(x) - f(x) f''(x)}{[f'(x)]^2} = 1 - \frac{f(x) f''(x)}{[f'(x)]^2}.$$

Substituting this back into  $\Phi'(x) = 1 - \frac{d}{dx} \left( \frac{f(x)}{f'(x)} \right)$  gives

$$\boxed{\Phi'(x) = \frac{f(x) f''(x)}{[f'(x)]^2}}.$$

At the true root  $x^*$ , we have  $f(x^*) = 0$ , hence

$$\Phi'(x^*) = 0.$$

This automatically satisfies the convergence condition  $|\Phi'(x^*)| < 1$ , and in fact implies *quadratic convergence*.

### 7.4 Practical Limitations

Despite its speed, Newton's method has pitfalls:

- If  $f'(x_k)$  is close to zero, the update explodes or diverges.
- If  $x_0$  is far from the root, the tangent may jump to a wrong region.
- Non-differentiable or oscillatory functions may cause chaotic behavior.

#### Note

In practice, Newton's method is often combined with *damping* or *line search* to ensure robustness:

$$x_{k+1} = x_k - \alpha_k \frac{f(x_k)}{f'(x_k)}, \quad 0 < \alpha_k \leq 1.$$

## 7.5 Example

Let  $f(x) = \cos x - x$ . Then

$$f'(x) = -\sin x - 1, \quad x_{k+1} = x_k - \frac{\cos x_k - x_k}{-\sin x_k - 1}.$$

Starting from  $x_0 = 1.0$ , we obtain:

| $k$ | $x_k$  | $f(x_k)$ | Error $ x_k - x^* $  |
|-----|--------|----------|----------------------|
| 0   | 1.0000 | -0.4597  | 0.2610               |
| 1   | 0.7503 | 0.0180   | 0.0112               |
| 2   | 0.7391 | 0.0001   | $1.4 \times 10^{-5}$ |
| 3   | 0.7391 | 0.0000   | $< 10^{-10}$         |

### Note

After only a few iterations, Newton's method reaches machine precision — a dramatic contrast to the slow, linear fixed-point iteration.

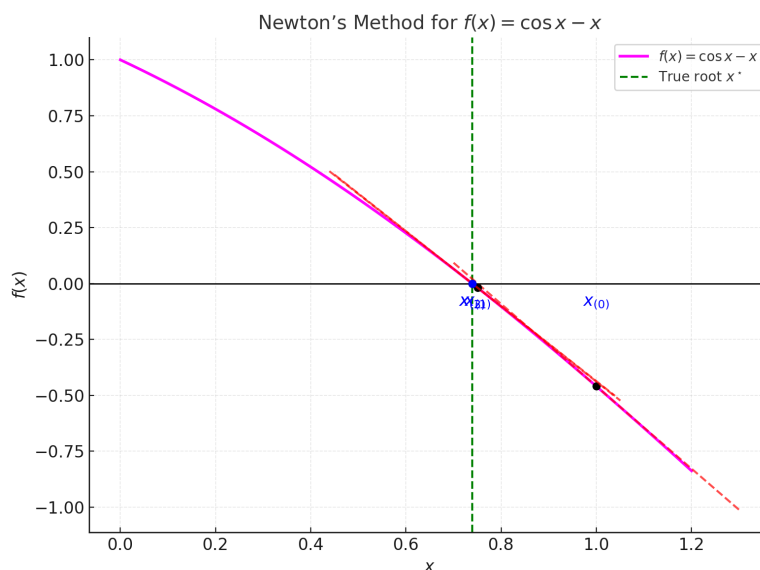


Figure: Newton's method applied to  $f(x) = \cos x - x$ . Each tangent intersects the  $x$ -axis, producing the next iterate  $x_{k+1}$ . Starting from  $x_0 = 1$ , the sequence converges rapidly to the true root  $x^* \approx 0.739$ .

## 7.6 Sensitivity to the Initial Guess

Newton's method converges very rapidly *if* the starting point is close to the root. However, if  $x_0$  is far away, the tangent may point in the wrong direction and jump to an entirely different region of the function.

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Although Newton's method is locally powerful, it is not globally reliable. The algorithm tends to move toward the *local minimum or maximum* of the function to which the initial guess  $x_0$  belongs. If the derivative changes sign or vanishes along the way, the tangent can point toward an entirely wrong region.

Consider  $f(x) = x^3 - 2x + 2$  with initial guess  $x_0 = 0$ . Here, the tangent at  $x_0$  leads to  $x_1 = 1$ , and the next tangent from  $x_1$  brings us back to  $x_2 = 0$ . The iteration becomes trapped in a two-cycle and never approaches the true root.

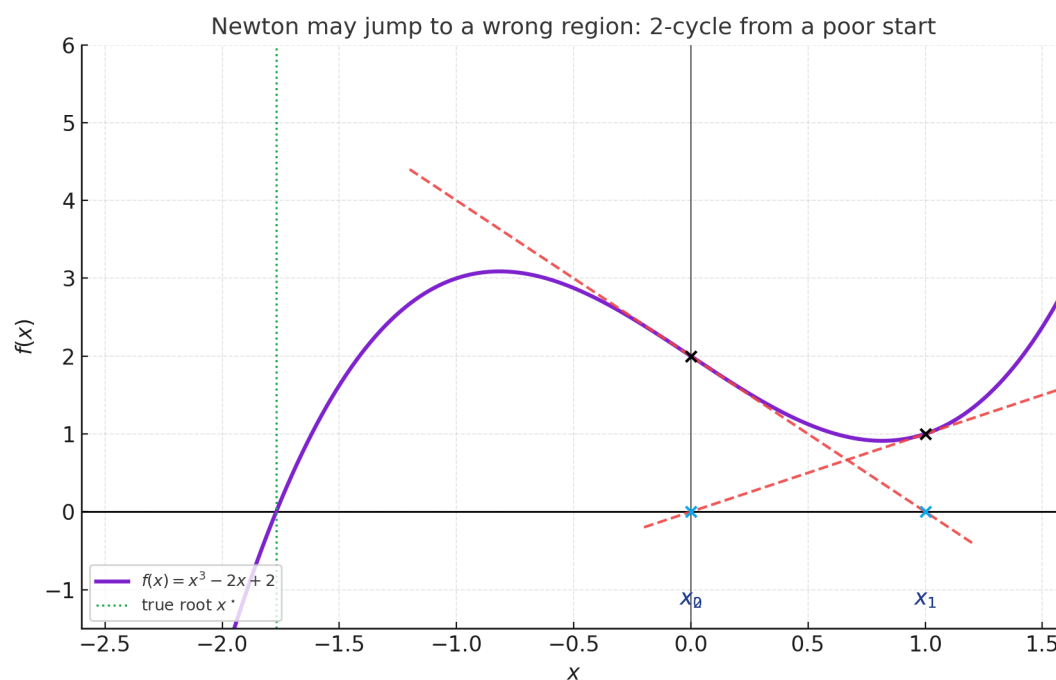


Figure: Newton with  $f(x) = x^3 - 2x + 2$ . With the poor initial guess  $x_0 = 0$ , the method falls into a two-cycle  $0 \leftrightarrow 1$  and fails to reach the true root  $x^* \approx -1.769$ .

### Note

Newton's method is locally powerful but not globally reliable. Good initial guesses are crucial; in practice, one often combines Newton's step with *bracketing* or *damping* to prevent wild jumps.

## 8 Multi-Point and Higher-Order Methods

So far, all iterative schemes have used *a single point*  $x_k$  to produce the next approximation. However, we can often improve performance by using information from *several previous iterates*. This leads to the family of **multi-point methods**.

### 8.1 Motivation

Newton's method achieves fast (quadratic) convergence, but it requires evaluating  $f'(x_k)$  at each step. In many real problems, especially when  $f$  is obtained from simulations or experiments, the derivative is either unavailable or too expensive to compute numerically.

#### Note

A natural question arises: can we approximate the derivative using previous iterates, and still retain the fast convergence behavior of Newton's method?

### 8.2 The Secant Method

The **Secant method** answers exactly that question. It replaces the true derivative  $f'(x_k)$  in Newton's update with a finite difference computed from the last two iterates:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Substituting this into Newton's formula gives:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

**Why this formula makes sense.** The key idea is geometric: if we connect the last two points  $(x_{k-1}, f(x_{k-1}))$  and  $(x_k, f(x_k))$  by a straight line, that line is our local approximation of the curve. The intersection of this secant with the  $x$ -axis provides the next approximation  $x_{k+1}$ .

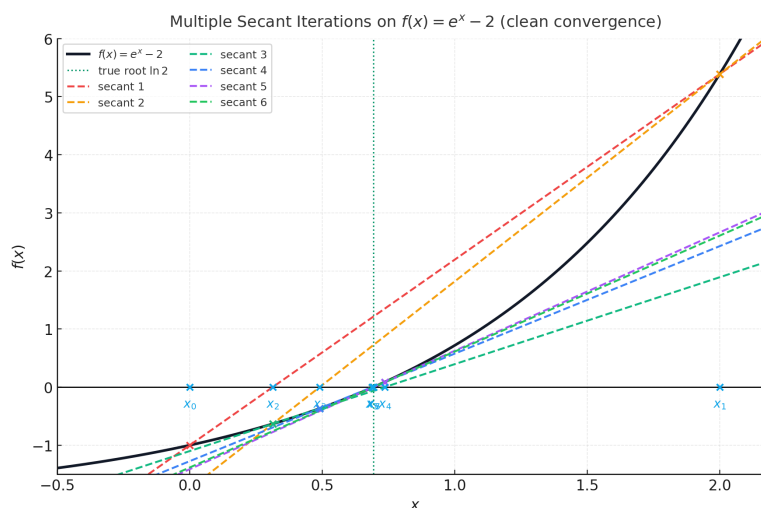


Figure: Successive Secant steps on  $f(x) = e^x - 2$  with  $x_0 = 0$ ,  $x_1 = 2$ . Each dashed line is a new secant using the two latest points; the intersections  $x_{k+1}$  approach the true root  $\ln 2$  cleanly.

Mathematically, the secant through  $(x_{k-1}, f(x_{k-1}))$  and  $(x_k, f(x_k))$  is

$$y = f(x_k) + \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}} (x - x_k).$$

Setting  $y = 0$  gives

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})},$$

which is exactly the update formula.

This rule is *consistent*: if  $x_k = x^*$  is a true root ( $f(x^*) = 0$ ), then  $x_{k+1} = x^*$ . Moreover, as  $x_{k-1} \rightarrow x_k$ , the finite difference approaches  $f'(x_k)$ , so the Secant method smoothly limits to Newton's method.

**Order of convergence.** Let the true root be  $x^*$ , and define the local error  $e_k = x_k - x^*$ . Near the root, we can approximate the function by its Taylor expansion:

$$f(x) \approx f'(x^*)(x - x^*) + \frac{1}{2}f''(x^*)(x - x^*)^2.$$

Substituting this approximation into the secant update and simplifying the resulting algebra (details omitted), one obtains that the next error behaves approximately as

$$e_{k+1} \approx C e_k e_{k-1},$$

where  $C$  is a constant depending on  $f''(x^*)$  and  $f'(x^*)$ .

Now, assume the errors follow an asymptotic pattern  $|e_{k+1}| \approx \lambda |e_k|^p$  for some order  $p > 1$ . Using the recursive form above and canceling constants gives

$$|e_{k+1}| \approx C |e_k| |e_{k-1}| \quad \Rightarrow \quad |e_k|^p \propto |e_k| |e_{k-1}| \quad \Rightarrow \quad p^2 = p + 1.$$

Solving this quadratic yields

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.618,$$

the famous **golden ratio**. This means that the Secant method converges *superlinearly*: faster than linear schemes (like fixed-point iteration), but slightly slower than Newton's quadratic convergence.

#### Note

The Secant method typically converges faster than fixed-point iteration but slightly slower than Newton: it achieves an *order of convergence*  $p \approx 1.618$  (the golden ratio), halfway between linear and quadratic.



### 8.3 Beyond Two Points - Optional

The Secant method uses the last two iterates to approximate the derivative. In principle, nothing prevents us from using *three*, *four*, or even more previous points to construct a higher-order polynomial approximation of  $f$ .

This leads to a family of **multi-point methods**, where

$$f'(x_k) \approx (\text{slope of interpolating polynomial through several past points}).$$

Each extra point increases the approximation accuracy of the derivative, and consequently, the *order of convergence* can exceed 2.

#### Note

In practice, using more than two points brings diminishing returns:

- the formulas become algebraically complex,
- rounding errors may grow,
- and extra evaluations of  $f$  often cost more than they save.

Hence, the Secant method strikes a sweet balance: no derivatives, only one extra function call per step, and a convergence rate  $p \approx 1.618$ .

**Example: using three points.** Suppose we keep the last three iterates  $x_{k-2}, x_{k-1}, x_k$ . Instead of fitting a straight line (as in the Secant method), we can fit a *quadratic polynomial*  $p_2(x)$  through the three points:

$$p_2(x_i) = f(x_i), \quad i = k-2, k-1, k.$$

We then use the intersection of this parabola with the  $x$ -axis as the next iterate:

$$x_{k+1} = \text{root of } p_2(x) = 0.$$

Geometrically, we are now approximating the curve of  $f(x)$  with a parabola instead of a line, leading to a better local model and faster convergence.

Formally, if we interpolate the inverse function  $x = \phi(f)$  through the last three points  $(x_{k-2}, f_{k-2})$ ,  $(x_{k-1}, f_{k-1})$ ,  $(x_k, f_k)$ , the next iterate is

$$x_{k+1} = \frac{f_{k-1}f_k}{(f_{k-2} - f_{k-1})(f_{k-2} - f_k)}x_{k-2} + \frac{f_{k-2}f_k}{(f_{k-1} - f_{k-2})(f_{k-1} - f_k)}x_{k-1} + \frac{f_{k-2}f_{k-1}}{(f_k - f_{k-2})(f_k - f_{k-1})}x_k.$$

This update is known as the **inverse quadratic interpolation formula**. It generalizes the Secant method (two points) and achieves an order of convergence around  $p \approx 2^{**}$  ( $p \approx 1.839$ ) faster, yet still derivative-free.

(\*\*): The local error satisfies  $e_{k+1} \propto e_k e_{k-1} e_{k-2}$ , which leads to the characteristic equation

$$p^3 = p^2 + p + 1,$$